

CASE STUDY

QuickMeds – Online Medicine Delivery System

Feature Focus: Shopping Cart & Order Management System

Student Name	Tanmay Warthe
Class	CT-B
Roll Number	178
Project Type	Front-End Web Application
Technology Stack	HTML5, CSS3, JavaScript, LocalStorage API

1. Introduction

QuickMeds is a web-based Online Medicine Delivery System developed as a research paper implementation assignment. The platform enables users to browse a catalogue of medicines, search for specific products, manage a shopping cart, and place orders — all from within a browser, with no server-side dependency.

The project draws its conceptual foundation from a research paper published in IRJMETs (International Research Journal of Modernization in Engineering Technology and Science), titled 'Online Medicine Delivery System / E-Pharmacy Management System'. The front-end implementation translates the proposed system architecture into a functional, user-accessible prototype.

This case study specifically analyses the Shopping Cart and Order Management feature of QuickMeds, examining its design rationale, technical implementation, and real-world applicability.

2. Problem Statement

Access to essential medicines remains a challenge in many urban and semi-urban areas of India due to store timings, physical distance, and the difficulty of tracking medicine availability. Traditional pharmacy visits require the customer to be present in person, which is often inconvenient for elderly patients, working professionals, or people with mobility limitations.

Existing e-commerce platforms either do not specialise in medicines or present a complex user interface that is not beginner-friendly. There is a clear gap for a dedicated, lightweight, and easy-to-use online pharmacy interface that allows users to:

- Browse and search medicines without requiring an account.
- Add multiple items to a cart and modify quantities in real time.
- Review a complete order summary with taxes and shipping before placing an order.
- Experience a smooth, responsive interface on both desktop and mobile devices.

3. Objectives

The primary objectives of the QuickMeds project are:

1. To develop a clean, responsive multi-page pharmacy website using HTML5, CSS3, and JavaScript.
2. To implement a fully functional product catalogue with search and filter capabilities.
3. To build a persistent shopping cart using the browser's LocalStorage API so that cart data survives page navigation.
4. To calculate and display a dynamic order summary including subtotal, 5% GST, and a fixed shipping charge.
5. To provide a user authentication interface (Sign In / Sign Up) with client-side form validation.
6. To ensure mobile responsiveness and cross-browser compatibility throughout the application.

4. Proposed Solution

QuickMeds proposes a front-end-only web application structured across four linked HTML pages. The solution deliberately avoids any back-end dependency, making it deployable on any static file host or directly from a local file system — ideal for a prototype or academic demonstration.

The core of the proposed solution is the LocalStorage-backed cart engine. When a user clicks 'Add to Cart' on any product card, the item is serialised as a JSON object and written to the browser's LocalStorage. The cart page reads this data on load, renders each item dynamically, and recalculates totals whenever the user modifies a quantity or removes a product. This architecture ensures data persistence across page refreshes and navigation without any server round-trips.

The solution also incorporates a cross-page search mechanism: the home page writes the search term to LocalStorage before redirecting to the products page, which reads and applies it as a filter on load.

5. Features

QuickMeds includes the following core features:

- **Product Catalogue** – Eight medicine products displayed in a responsive grid layout with name, description, and price.
- **Search Functionality** – A search bar on the home page filters products on the products page using LocalStorage as a cross-page relay.
- **Add to Cart** – Each product card has an 'Add to Cart' button. Re-adding an existing item increments its quantity instead of creating a duplicate entry.
- **Cart Management** – Users can increase or decrease item quantities using +/- controls, or remove items entirely. The cart UI updates in real time.
- **Order Summary** – The cart page calculates and displays subtotal, fixed shipping (₹40), 5% GST, and the grand total, all updating dynamically.
- **Checkout Flow** – A 'Proceed to Checkout' button confirms the order, clears the cart, and redirects to the home page.
- **User Authentication UI** – A dual-panel Login page with Sign In and Sign Up forms, including client-side password match validation.
- **Responsive Design** – The layout adapts to mobile, tablet, and desktop screens using CSS Flexbox and Grid.

6. System Modules

The application is structured into four interconnected modules, each corresponding to an HTML page:

6.1 Home Module (index.html)

Serves as the landing page. Presents the QuickMeds branding banner, a search bar, and a 'Featured Products' section showing four products. Handles the search initiation logic that stores the query in LocalStorage and redirects to the products page.

6.2 Product Catalogue Module (products.html)

Displays the complete inventory of eight medicines in a card grid. On page load, checks LocalStorage for a pending search term and filters the visible cards accordingly. Each card's 'Add to Cart' button invokes the shared cart utility function.

6.3 Cart & Order Module (cart.html)

The most feature-rich module. Reads cart data from LocalStorage and dynamically renders each cart item with quantity controls and a remove button. Computes and displays the live order summary. Handles the checkout action by clearing LocalStorage and confirming the order to the user.

6.4 Authentication Module (login.html)

Provides a side-by-side Sign In and Sign Up form layout. Sign In redirects to the home page on submission. Sign Up validates that passwords match before confirming account creation. Both forms use HTML5 validation attributes for field-level checks.

7. Feature Deep Dive – Shopping Cart & Order Management

The Shopping Cart is the most technically significant feature of QuickMeds. Below is a step-by-step walkthrough of its implementation.

7.1 Data Storage Architecture

Cart data is stored as a JSON array in the browser's LocalStorage under the key 'cart'. Each element in the array is an object with three properties:

Property	Description
name	String – the product name (used as the unique identifier)
price	Number – unit price of the product in Indian Rupees (₹)
quantity	Number – the number of units the user intends to purchase

7.2 Add to Cart Logic

When the user clicks 'Add to Cart', the `addToCart()` function is called with the product name and price as arguments. The function: (1) reads the existing cart array from `LocalStorage`, (2) searches for an entry with a matching name, (3) increments the quantity if found, or pushes a new object with quantity 1 if not found, and (4) writes the updated array back to `LocalStorage`. An alert confirms the action to the user.

7.3 Cart Rendering

The `loadCart()` function on `cart.html` reads the cart array on page load. If the array is empty, an 'empty cart' message with a 'Browse Products' call-to-action is rendered. If items exist, each is rendered as a cart-item div containing the product name, unit price, quantity controls, line total, and a Remove button.

7.4 Quantity Management

The `updateQuantity(index, change)` function adds the change value (+1 or -1) to the item at the given index. If the resulting quantity drops to zero or below, the item is removed from the array using `splice()`. The cart is then saved back to `LocalStorage` and `loadCart()` is called again to re-render the UI, keeping the view always in sync with the data.

7.5 Order Summary Calculation

The `updateSummary()` function iterates over the cart array and computes:

Field	Calculation
Subtotal	Sum of (price × quantity) for all items
Shipping	₹40 flat (₹0 if cart is empty)
GST (5%)	5% of the Subtotal
Grand Total	Subtotal + Shipping + GST

All four values are updated in the DOM in real time after every quantity change or item removal.

7.6 Checkout

The `proceedToCheckout()` function first verifies that the cart is not empty. If items exist, it shows a success alert, clears the 'cart' key from `LocalStorage`, and redirects the user to the home page — simulating a completed order placement.

8. Technology Stack

Technology	Purpose
HTML5	Page structure and semantic markup across all four pages
CSS3	Styling, Flexbox/Grid layout, responsive breakpoints, animations
JavaScript (ES6+)	Cart logic, DOM manipulation, <code>LocalStorage</code> API interaction
<code>LocalStorage</code> API	Client-side persistent storage for cart items and search queries
Google Fonts (Poppins)	Typography for a modern, readable UI
CSS Custom Properties	Consistent colour theming across the entire application

9. Working of the System

The user journey through QuickMeds follows a linear, intuitive flow:

7. The user opens index.html and is greeted by the QuickMeds banner and featured products.
8. The user types a medicine name into the search bar and clicks 'Search'. The term is saved to LocalStorage and the user is redirected to products.html.
9. On products.html, matching products are displayed (non-matching cards are hidden). The user clicks 'Add to Cart' on one or more products.
10. The user navigates to cart.html. All previously added items appear with their quantities and individual totals.
11. The user adjusts quantities using the +/- buttons. The order summary updates instantly.
12. The user clicks 'Proceed to Checkout'. An order confirmation message is shown, the cart is cleared, and the user is returned to the home page.

10. Advantages

- No Installation Required – Runs entirely in the browser; no server, database, or runtime environment needed.
- Persistent Cart – LocalStorage ensures cart contents are retained even if the user refreshes the page or navigates away.
- Real-Time Updates – Order summary recalculates immediately on every user interaction, providing instant feedback.
- Lightweight & Fast – No external API calls or heavy frameworks; all interactions are synchronous and near-instantaneous.
- Responsive Design – The Flexbox and Grid-based layout adapts seamlessly to screens of all sizes.
- Beginner-Friendly UI – Clean visual hierarchy, clear call-to-action buttons, and colour-coded states make the interface easy to use.

11. Limitations

- Front-End Only – No real order processing, payment gateway, or inventory management exists; data is simulated.
- No User Authentication Backend – The login form validates only on the client side; no actual user accounts are stored or verified.
- LocalStorage Constraints – Cart data is browser-specific and device-specific; it cannot be shared across devices or sessions.
- No Prescription Upload – A real pharmacy system would require prescription verification for certain medicines, which is absent here.
- Limited Product Catalogue – Only eight hardcoded products are available; there is no admin panel to add or manage inventory.
- No Stock Management – The system does not track or enforce product availability limits.

12. Future Scope

The following enhancements are proposed to evolve QuickMeds from a prototype into a production-ready system:

13. Back-End Integration – Connect to a Node.js / PHP / Python backend with a relational database (MySQL / PostgreSQL) for real order processing and inventory management.
14. User Account System – Implement secure user registration, login with JWT/session-based authentication, and order history tracking.
15. Payment Gateway – Integrate Razorpay, PayU, or Stripe for live payment processing.
16. Prescription Upload – Add a file upload module allowing users to attach a doctor's prescription for regulated medicines, with pharmacist review functionality.
17. Push Notifications – Notify users about order status updates, delivery ETAs, and promotional offers.
18. Admin Dashboard – A protected admin panel for managing product listings, pricing, stock levels, and processing incoming orders.
19. Progressive Web App (PWA) – Convert the site into a PWA for offline capability and mobile home-screen installation.

13. Conclusion

QuickMeds successfully demonstrates a complete front-end implementation of an Online Medicine Delivery System. The Shopping Cart and Order Management feature — the focal point of this case study — is built on a clean, well-structured LocalStorage architecture that delivers a smooth, real-time user experience without any server dependency.

The project fulfils all its stated objectives: product browsing, cross-page search, persistent cart management, dynamic order calculation, and a simulated checkout flow are all implemented and functional. The codebase is modular, with each page handling its own concerns while sharing a common design language and utility functions.

While the current implementation is intentionally scoped as an academic prototype, the design decisions made — separation of concerns, local persistence, responsive layout — provide a solid foundation for future full-stack development. QuickMeds demonstrates that a well-designed front-end prototype can effectively communicate the complete user journey of a complex real-world application.